Assignment:07 **Shaurya Singh(2402MC11)**

Q1 : Thread Creation and Shared Counter

The test program threadtest.c allocates a stack for each of three threads and starts a worker that increments a shared counter 100,000 times. The **key shared data structure is the counter.**

Result:

The thread test completed and printed a final counter of less than 300,000.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu
/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_str
essfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress use
r/_forphan user/_dorphan user/_sync user/_threadtest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blo
cks 1953 total 2000
balloc: first 1023 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M
-smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img
,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmi
o-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ threadtest
Final counter: 101616
Expected counter: 300000
$
```

Q2 : Semaphores and Producer-Consumer

In synctest.c, semaphore 0 protects the shared counter. Only a worker holding the semaphore performs counter++, so concurrent workers cannot enter that critical section at the same time.

Before/after race-condition comparison

Aspect	Unsynchronized counter (Q1)	Semaphore-protected counter (Q2)
Critical section	No mutual exclusion	Counter update done by sem_wait/sem_signal
Possible failure	Concurrent read–modify–write can lose increments	Only one worker updates counter at a time
Recorded result	300,000 in one run; not a guarantee	300,000 observed in the recorded run
Conclusion	Race is possible even if one run succeeds	Critical section is serialized in the tested design

Observed producer-consumer result

The test printed the expected synchronized counter of 300,000. The producer emitted items 1 through 10, and the consumer consumed the items in order.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu
ve file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,dr
ive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

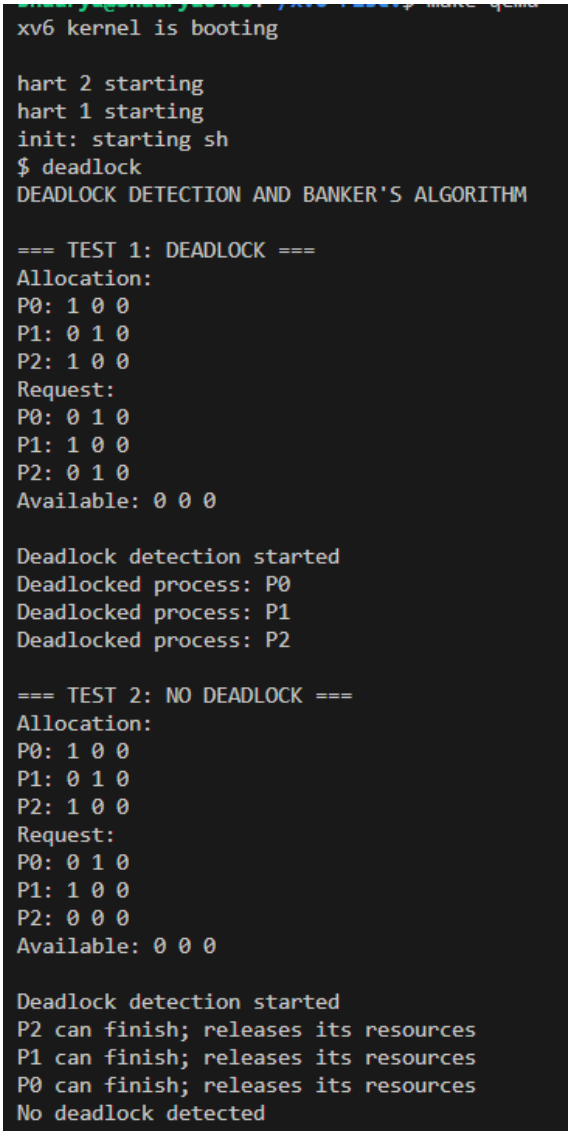
hart 2 starting
hart 1 starting
init: starting sh
$ synctest
synctest started
initializing semaphore
Synchronized counter: 300000
Expected counter: 300000
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Producer-Consumer completed
$
```

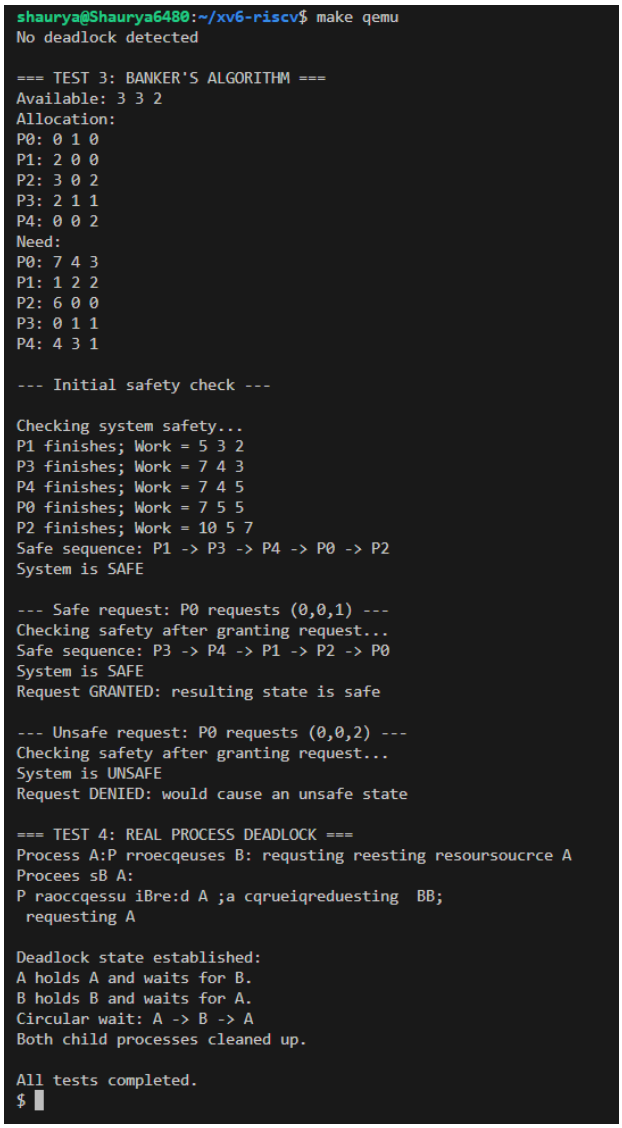
Q3 : Deadlock Detection and Banker's Algorithm

When no more processes can finish, any process still marked unfinished is reported as deadlocked.

Test case	Input / operation	Recorded output / conclusion
1. Deadlock state	P0, P1, P2; Available = (0,0,0)	P0, P1 and P2 reported deadlocked.
2. No-deadlock state	Same allocations; P2 Request = (0,0,0)	P2, then P1 and P0 finish; no deadlock detected.
3. Initial safety	Available = (3,3,2)	Safe sequence: P1 → P3 → P4 → P0 → P2.
4. Safe request	P0 requests (0,0,1)	Granted; resulting state reported safe.
5. Unsafe request	P0 requests (0,0,2)	Denied after safety check reports an unsafe state.

Observed Result:





Q4 : Priority Scheduling and Aging

The scheduler scans runnable processes and chooses a process with the lowest numeric priority. Aging uses elapsed timer ticks: after a runnable process has waited for the configured interval, its priority number is reduced by one, down to 1, to mitigate starvation.

PID	Initial priority	Final priority	Observed completion order	Aging observed
4	2	2	1st	No
5	5	5	2nd	No
6	10	9	3rd	Yes
7	20	19	4th	Yes

Observed Results:

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu CPUS=1
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bi
al 2000
balloc: first 1175 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kern
aphic -global virtio-mmio.force-legacy=false -drive file=
0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus

xv6 kernel is booting

init: starting sh
$ priotest
Priority Scheduler + Aging Test
Priority 1 = highest, 20 = lowest
Child PID 4 ready, initial priority 2
Child PID 5 ready, initial priority 5
Child PID 6 ready, initial priority 10
Child PID 7 ready, initial priority 20
All children ready; releasing workload
Child PID 4 starting CPU work
Child PID 4 completed, final priority 2
Child PID 5 starting CPU work
Child PID 5 completed, final priority 5
Child PID 6 starting CPU work
Child PID 6 completed, final priority 9
Child PID 7 starting CPU work
Child PID 7 completed, final priority 19
All children completed successfully.
$
```

Problems Faced and Resolutions

During the implementation of the assignment, several issues were encountered and resolved. In the threading experiment, the shared counter initially produced inconsistent results because multiple threads accessed and modified it simultaneously. This was resolved by using semaphores to synchronize access to shared resources. In the producer-consumer problem, synchronization was required to prevent buffer overflow and underflow and to ensure correct coordination between producers and consumers. While implementing deadlock detection and the Banker's algorithm, the main challenge was correctly calculating available resources, outstanding requests, and safe sequences; this was addressed by carefully checking the allocation, request, and need matrices for each test case. In the priority scheduling implementation, compilation errors and incorrect initialization of process fields were encountered. These were resolved by initializing the fields in allocproc(), correcting system call declarations and definitions, and removing unsupported kernel printing code. An additional issue occurred when launching QEMU because another instance was already using the file system image; closing the existing instance resolved the problem. Finally, the scheduling test was used to observe priority-based execution and aging, which gradually improves the priority of processes that wait for a long time.